Adding forms to your Express server

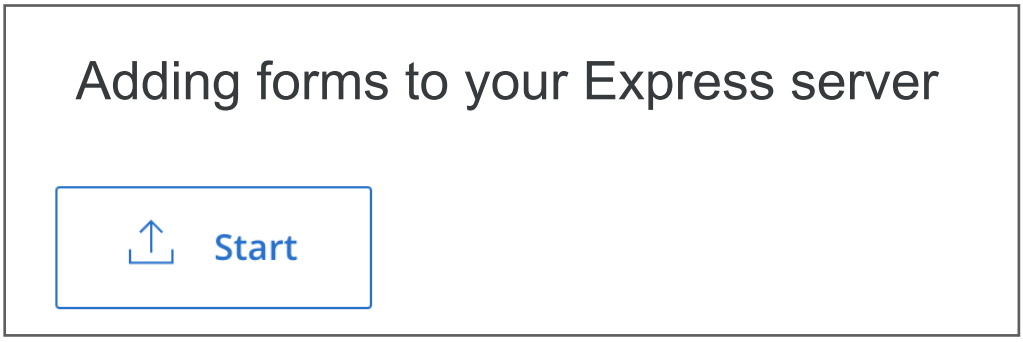
Welcome to this Lab activity

In this lab activity you will be exploring how to add forms to your Express web application. Forms are very important for data manipulation and allow you to collect, update and verify many information. For the purpose of this lab activity you will be working on the same structure as the 'Adding html to your Express server' lab but under a new folder called *topic4*.

Start the Lab environment application

It is simple to launch a lab exercise. You only need to click on the button "Start" below the activity title to enter a lab environment.

Let's explore this lab activity. Go ahead and click on the "Start" button!



Task 1: Build a new Express web server with an html file for each route

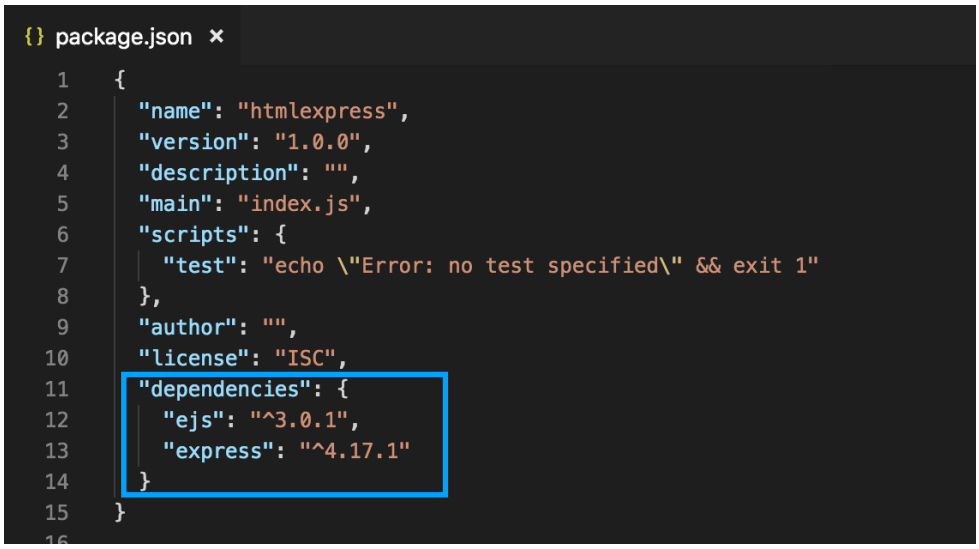
The folder structure has already been partially constructed for you and organised into different topics. For the purpose of this lab, you will be making changes inside the *topic4* folder structure. Let's get started!

Please do not delete or move any existing folders/files inside the lab environment.

Use the Visual Studio Code Terminal and run the following commands:

- **cd topic4/htmlExpress**: type this command and press *Enter*. This command will change your working directory to be the *htmlExpress* folder.
- **npm init**: type this command and press *Enter*. This command will set up a new or existing npm package. You can skip all the npm initialisation questions by pressing *Enter*. Once the npm package file has been created, you can view it inside the *htmlExpress* directory (*package.json*). The *package.json* file contains a set of information regarding your current node project.
- **npm install express --save**: type this command and press *Enter*. This command will install the *Express* framework within your working directory so that it will be available for you to use.
- **npm install ejs --save**: type this command and press *Enter*. This command will install the *EJS* framework within your working directory so that it will be available for you to use.

Once you run the above Terminal commands, your *package.json* file located inside the *topic4/htmlExpress* folder should contain both the **Express** and **EJS** modules:



The next step is to add some code to the *index.js* file located inside the *topic4/htmlExpress* folder in order to create your web server.

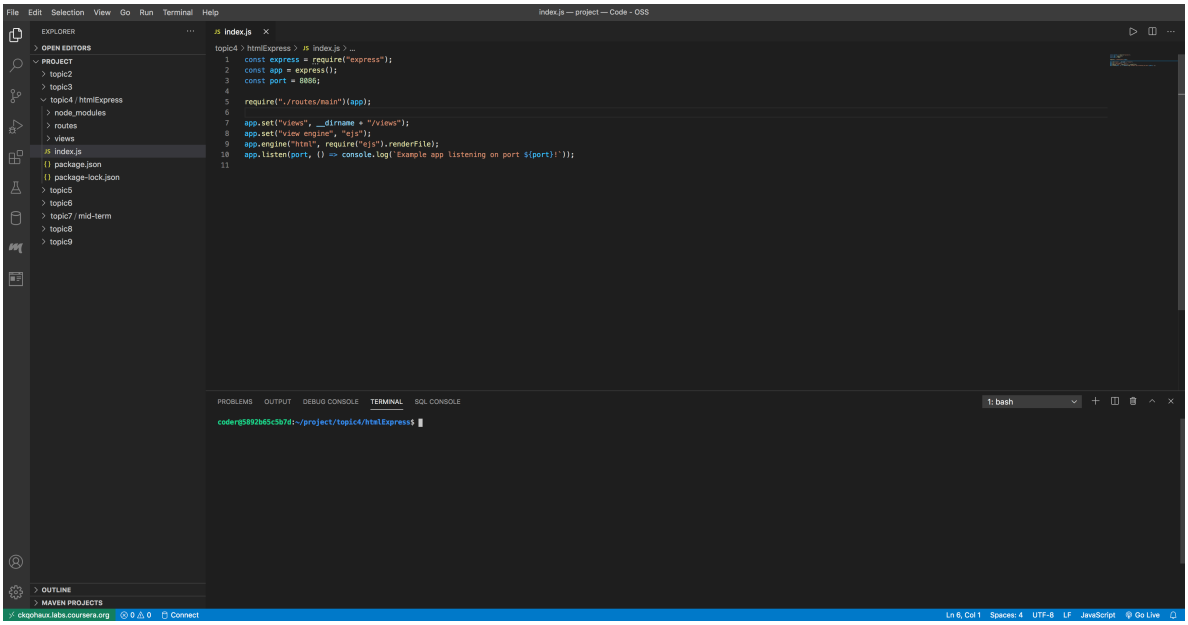
Add the following code to the *index.js* file and remember to save it:

```
const express = require ("express");
const app = express();
const port = 8086;

require("./routes/main")(app);

app.set("views",__dirname + "/views");
app.set("view engine", "ejs");
app.engine("html", require("ejs").renderFile);
app.listen(port, () => console.log(`Example app listening on port ${port}!`));
```

If you have correctly followed all the above steps, your environment should be similar to the one below:



Let's add the missing parts to both the *topic4/htmlExpress/routes and topic4/htmlExpress/views* folders to have your web application serve html pages.

Create three html files inside the *topic4/htmlExpress/views* folder called *index.html about.html* and *search.html* as you did inside the *topic3/htmlExpress/views* folder. Copy the content of each html page from the *topic3/htmlExpress/views* folder to the ones inside the *topic4/htmlExpress/views* folder.

Finally copy the code inside the *topic3/htmlExpress/routes/main.js* file inside the *topic4/htmlExpress/routes/main.js* file.

At this point you should have a copy of your Express application created in topic3 inside the *topic4/htmlExpress* folder. In your application there is no user interaction apart from calling different URLs but what about adding a form to your “search.html” file?

Add the following code before the closing tag </body> to your “search.html” file located inside the *topic4/htmlExpress/views folder:*

```
<form action="/search" method="GET">
  <input id="search-box" type="text" name="search-keyword" value="Default">
  <input type="submit" value="OK" >
</form>
```

Running the index.js file

Run the *index.js* file with the following Terminal command:

- **node index.js**: type this command and press Enter. The node command, followed by the file name, tells Node.js to execute the content of the file.

The above command will start a web server running on port 8086.

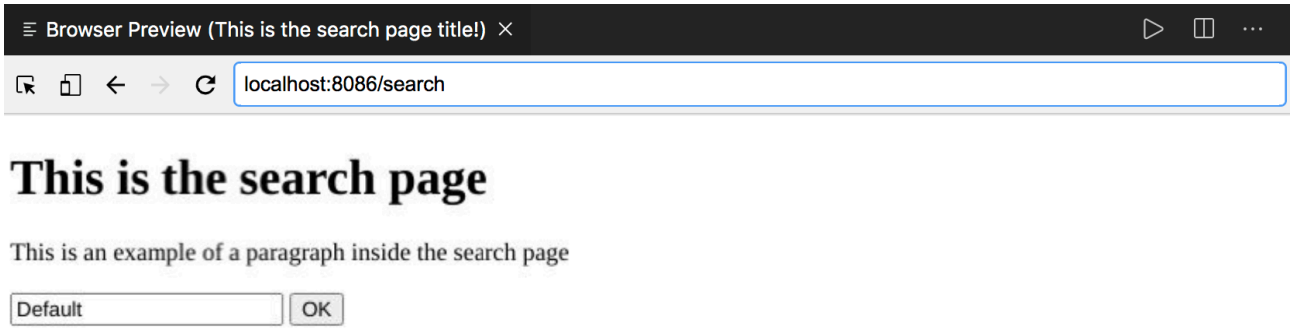
Task 2: Access your server via HTTP

Now that your code is running, serving your application on port 8086, you can access your web page from a browser!

Use the “Browser Preview” plugin to visualise your web application.

If you do not remember how to use the “Browser Preview” plugin, please refer to the “Creating my first Node.js web server” lab instructions.

Type *localhost:8086/search* on the “Browser Preview” tab and press *Enter* on your keyboard to visualise your web application “search” page:



End of Section

Congratulations for completing this section. As long as you have saved your work, your files will remain when you close this lab activity so do not worry about losing your data. You have successfully created a web application that serves html files containing forms. How can you retrieve the search keyword entered by the user of your web application? We will come back to this question in future lab activities.